ARM PrimeCell Driver

UART v3.0 (PL011/PL010)

Test Report

PL010-TREP-1000-B

Open Access

Proprietary notice

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	B
PrimeCell driver identifier	UART
Code Version	3.0
PrimeCell identifier	PL011/PL010
Test performed	12 July 2001

Contents

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A		First release of test report
B	12/0701	Updated for v3.0

2 RELEASED FILES

PL010_ISPC_1000	API document
PL010–TREP-1000	Test Results document
apUART/apUART.h	Public header file
apUART/UART.h	Private header file
apUART/UART.c	Source code
apUART/UARTtst.c	Released test code

3 TEST ENVIRONMENT

3.1 Test Specification Summary

	Specification (Intended)	**Actual Test Environment** **If different from Specification**
Code Test Platform	INTEGRATOR	
Core used for test	7TDMI	940T
Module ROM size (RO code + RO data)	3596 Bytes	3720 Bytes Integrator AP 3920 Bytes PL010 variant 4844 Bytes PL011 variant
Module RAM size (ZI data) per instance	116 Bytes	60 Bytes
PrimeCell tested	PL011/PL010	PL011 / Integrator variant
External hardware used	N/A	
Performance obtained	N/A	
Driver modules	UART.c	
Test modules	uarttst.c uarttstcalls.c	
Standard modules required (normal archive location)	os.c, oss.s, vic.c, vicdispatch.s, boot.s, timer.c, timertst.c, vectors.s	os.c, oss.s, main.c, test.c, boot.s, inttst.c, int.c, dispatch.s vectors.s, timer.c, timertst.c
Custom modules required (in [project]/privates/ UART**)**	applatfm.h, os.c, test.c, uarttstcalls.c, vectors.s,	applatfm.h, uarttstcalls.c

The test modules can be used to communicate with an external terminal – to do this the comment characters in front of **`#define apUART_TerminalTest`** must be removed and placed in front of **`#define apUART_UARTTest`**.

3.2 Hardware requirements

3.2.1 FPGA

Is the image used to program the FPGA archived YES

If so, at what location N/A

3.2.2 Test configuration

Is the hardware configuration documented YES

If so, at what location N/A

PL011:

Single PL011 situated in FPGA

PL010 (Integrator variant):

Standard Integrator set-up – connection between serial port 0
and HyperTerminal for terminal testing (57600, 8 – N – 1).

3.2.3 External connectors required

3.2.4 Devices on prototyping area

3.2.5 External test equipment

HyperTerminal configured to 57600 baud, 8 bits, no parity, 1 stop bit.

4 CODE COVERAGE

4.1 Overall

Code coverage tests have been performed. YES

Code coverage details below

Code coverage using uart.mcp and uarttst.c.

```
Code Coverage report
=====================


Routine returns:
Missed: CC(R,0) CC(R,1) CC(R,2) CC(R,5) CC(R,6) CC(R,9) CC(R,16) CC(R,22) CC(R,25) CC(R,28)
CC(R,30) CC(R,31) CC(R,34) CC(R,37) CC(R,38) CC(R,39) CC(R,41) CC(R,42) CC(R,43) CC(R,45)
CC(R,47) CC(R,48) CC(R,50) CC(R,52) CC(R,53) CC(R,54) CC(R,55) CC(R,56) CC(R,57) CC(R,58)
CC(R,59) CC(R,61) CC(R,62) CC(R,64) CC(R,65) CC(R,66) CC(R,69) CC(R,74) CC(R,77) CC(R,82)
CC(R,85) CC(R,88) CC(R,91) CC(R,94) CC(R,97) CC(R,100) CC(R,103) CC(R,106) CC(R,112)
CC(R,115) CC(R,118) CC(R,119) CC(R,120) CC(R,122) CC(R,123)
Code coverage: 55 %


Blocks:
Missed: CC(B,0) CC(B,1) CC(B,2) CC(B,3) CC(B,4) CC(B,5) CC(B,12) CC(B,13) CC(B,14) CC(B,15)
CC(B,23) CC(B,26) CC(B,28) CC(B,29) CC(B,36) CC(B,38) CC(B,43) CC(B,45) CC(B,46) CC(B,47)
CC(B,48) CC(B,55) CC(B,57) CC(B,59) CC(B,64) CC(B,65) CC(B,66) CC(B,68) CC(B,69) CC(B,70)
CC(B,72) CC(B,73) CC(B,74) CC(B,76) CC(B,77) CC(B,81) CC(B,82) CC(B,83) CC(B,85) CC(B,86)
CC(B,88) CC(B,89) CC(B,90) CC(B,92) CC(B,93) CC(B,94) CC(B,95) CC(B,96) CC(B,97) CC(B,98)
CC(B,99) CC(B,100) CC(B,101) CC(B,102) CC(B,104) CC(B,105) CC(B,107) CC(B,108) CC(B,109)
CC(B,110) CC(B,113) CC(B,114) CC(B,119) CC(B,120) CC(B,121) CC(B,122) CC(B,123) CC(B,124)
CC(B,125) CC(B,126) CC(B,127) CC(B,131) CC(B,134) CC(B,136) CC(B,137) CC(B,138) CC(B,139)
CC(B,140) CC(B,142) CC(B,143)
Code coverage: 44 %
```

Code coverage using testuart.mcp and uarttstcalls.c.

```
Code Coverage report
=====================


Routine returns:
Missed: CC(R,0) CC(R,1) CC(R,2) CC(R,5) CC(R,6) CC(R,9) CC(R,16) CC(R,22) CC(R,25) CC(R,28)
CC(R,31) CC(R,34) CC(R,37) CC(R,38) CC(R,39) CC(R,41) CC(R,42) CC(R,43) CC(R,45) CC(R,47)
CC(R,48) CC(R,50) CC(R,52) CC(R,55) CC(R,57) CC(R,66) CC(R,69) CC(R,74) CC(R,77) CC(R,82)
CC(R,85) CC(R,88) CC(R,91) CC(R,94) CC(R,97) CC(R,100) CC(R,103) CC(R,106) CC(R,112)
CC(R,115)
Code coverage: 67 %
```

```
Blocks:
Missed: CC(B,0) CC(B,1) CC(B,2) CC(B,3) CC(B,4) CC(B,5) CC(B,12) CC(B,13) CC(B,14) CC(B,15)
CC(B,26) CC(B,28) CC(B,29) CC(B,36) CC(B,38) CC(B,43) CC(B,45) CC(B,46) CC(B,47) CC(B,48)
CC(B,55) CC(B,57) CC(B,59) CC(B,64) CC(B,65) CC(B,66) CC(B,69) CC(B,70) CC(B,72) CC(B,73)
CC(B,74) CC(B,76) CC(B,77) CC(B,81) CC(B,82) CC(B,83) CC(B,88) CC(B,89) CC(B,90) CC(B,92)
CC(B,94) CC(B,97) CC(B,100) CC(B,109) CC(B,110) CC(B,113) CC(B,114) CC(B,119) CC(B,120)
CC(B,121) CC(B,122) CC(B,123) CC(B,124) CC(B,125) CC(B,126) CC(B,127) CC(B,136) CC(B,137)
Code coverage: 59 %
```

Percentage of normal routine exits tested. 67 %

- *Examine all routine exits listed as untested to ensure that these are not used in normal flow.*

All untested routine exits examined. YES

Percentage of blocks tested. 59 %

4.2 Untested Blocks and Routine Exits

Blocks and routine exits not covered by these tests have been examined and categorised as below.

4.2.1 General cases

The block numbers, as generated by the code coverage tool, are listed below.

Reason	Block numbers	Routine numbers
Block is switch statement (end not reached due to break)		
Block end never reached due to break/return statement(s)	CC(B,48) CC(B,57) CC(B,59) CC(B,64) CC(B,65) CC(B,66) CC(B,69) CC(B,70) CC(B,89) CC(B,109) CC(B,110) CC(B,113) CC(B,114) CC(B,119) CC(B,120) CC(B,121) CC(B,122) CC(B,123) CC(B,124) CC(B,125) CC(B,126) CC(B,127) CC(B,136) CC(B,137)	CC(R,9) CC(R,16) CC(R,22) CC(R,25) CC(R,28) CC(R,31) CC(R,34) CC(R,47) CC(R,66) CC(R,69) CC(R,74) CC(R,77) CC(R,82) CC(R,85) CC(R,88) CC(R,91) CC(R,94) CC(R,97) CC(R,100) CC(R,103) CC(R,106) CC(R,112) CC(R,115)
Block implements a trivial range check e.g. `if (x<4) {x=4;}`		
Default statement in switch not expected to be executed		
Debug Code never executed	CC(B,0) CC(B,1) CC(B,2) CC(B,3) CC(B,4) CC(B,5) CC(B,45) CC(B,46) CC(B,92)	CC(R,0) CC(R,1) CC(R,2) CC(R,50)
Code never executed – included for safety in cases of future expansion: apUART_StateSizeGet and apUART_RawISR	CC(B,12) CC(B,13) CC(B,14) CC(B,15)	CC(R,5) CC(R,6)
	25%	23%

4.2.2 Special cases

Block number	Routine number	Routine	Reason
B,26		apUART_IntHandler	If there is a RECEIVE interrupt, all the data is read and the read doesn't end on a word boundary this block is called as a buffer flush.
B,28 B,29		apUART_IntHandler	If there is a RECEIVE interrupt and no receive action is currently underway this block is called.
B,36		apUART_IntHandler	Receive overrun interrupt
B,38		apUART_IntHandler	Parity error interrupt
B,43		apUART_IntHandler	Invalid transmit interrupt
B,47		apUART_Initialize	Empty receive FIFO
B,55		apUART_BaudRateSet	Specific test in baud rate setting not expected in normal flow.
B,72 B,73 B,74	R,37 R,38 R,39	apUART_Transmit	Error returns if incorrect entry conditions, not expected in normal flow.
B,67 B,77		apUART_Transmit	Check for non word aligned buffer
B,81 B,82 B,83	R,41 R,42 R,43	apUART_Receive	Error returns if incorrect entry conditions not expected in normal flow.
B,88	R,45	apUART_Continue	Error return if incorrect entry condition, not expected in normal flow.
B,90	R,48	apUART_Read	Error return if incorrect entry condition, not expected in normal flow.
B,94	R,52	apUART_Write	Error return if incorrect entry condition, not expected in normal flow.
B,97	R,55	apUART_Read_N	Error return if incorrect entry condition, not expected in normal flow.
B,100	R,57	apUART_Write_N	Error return if incorrect entry condition, not expected in normal flow.
15%	9%		

5 RELEASED TEST MODULE

5.1 Overview

The released test module is named **`UARTtst.c`**.

It is used as a self-test module for the **`UART`** PrimeCell and also as a part of the formal testing.

Test environment is as described earlier.	YES
Code coverage: Percentage of blocks tested by this test module.	44%

5.2 Changes to test environment

None for public tests, UART0 used, this can be altered in platfm.h if UART2 used for serial port on logic module.

The tests involve performing an internal loop back test on a single device if UART_UARTTEST defined in uarttst.c this is the default.

The serial ports on an Integrator suffice with connection via a standard 9-pin serial cable between Serial A (UART0) or Serial B (UART1) to a PC for UART_TERMINALTEST.

5.3 Outline

The public test code is not designed to exercise all of the UART API. It tests some of the API calls by writing and reading settings to and from memory and then performs a transmit and receive action which aim to confirm the UART is correctly connected to the interrupt controller.

5.4 Initialisation

5.4.1 apUART_SelfTest()

⊕ Call to `apUART_Initialise()` function to set up peripheral base address and interrupts.

▪ Number of interrupts used	1
▪ Specific parameters passed in configuration data	Rx FIFO size (not used) UART Clock Frequency (#define'd to 14745600)

5.5 UART_APICallsTest

The following API tests call a subset of the public API functions to provide some testing of setting and retrieving register configuration values.

5.5.1 UART_BaudRateTest

This tests the configuration of the UART's baud rate settings via the setting and retrieving of three different baud rates. After each baud value is set the value is retrieved and compared to the set value. If all the values match the test is identified as being successful.

The two public function calls used are:

⊕ apUART_BaudRateSet(apOS_UART_oId oId, UWORD32 Rate)

⊕ apUART_BaudRateGet(apOS_UART_oId oId)

where the parameter values are:

- oId – apOS_UART_0.
- Rate - 2400, 57600 & 115200 are the three different baud values used within this test.

```
Baud rate tests completed successfully: Pass
```

5.5.2 UART_DCDTest

This tests the configuration of the Data Carrier Detect (DCD) output on the UART. For the PL010 variant the test checks that both functions return apERR_UNSUPPORTED, the PL011 version checks to ensure the retrieved value of the DCD configuration is the same as the set value.

The two public function calls used are:

⊕ apUART_DCDConfigSet(apOS_UART_oId oId, apUART_eDCDEnable eDCDflag)

⊕ apUART_DCDConfigGet(apOS_UART_oId oId, apUART_eDCDEnable *peDCDflag)

where the parameter values are:

- oId – apOS_UART_0.
- eConfig - apUART_DCD_ENABLED.

PL010:

```
DCD configuration tests completed successfully - informed that DCD is not supported: Pass
```

PL011:

```
DCD configuration tests completed successfully: Pass
```

5.5.3 UART_RTSTest

This tests the configuration of the Request To Send (RTS) output on the UART. For the PL010 variant the test checks that both functions return apERR_UNSUPPORTED, the PL011 version checks to ensure the retrieved value of the RTS configuration is the same as the set value.

The two public function calls used are:

⊕ apUART_RTSConfigSet(apOS_UART_oId oId, apUART_eRTSEnable eRTSflag)

⊕ apUART_RTSConfigGet(apOS_UART_oId oId, apUART_eRTSEnable *peRTSflag)

where the parameter values are:

- oId – apOS_UART_0.
- eRTSflag - apUART_RTS_ENABLED.

PL010:

```
RTS configuration tests completed successfully - informed that RTS is not supported: Pass
```

PL011:

```
RTS configuration tests completed successfully: Pass
```

5.5.4 UART_DTRTest

This tests the configuration of the Data Terminal Ready (DTR) output on the UART. For the PL010 variant the test checks that both functions return apERR_UNSUPPORTED, the PL011 version checks to ensure the retrieved value of the DTR configuration is the same as the set value.

The two public function calls used are:

⊕ apUART_DTRConfigSet(apOS_UART_oId oId, apUART_eDTREnable eDTRflag)

⊕ apUART_DTRConfigGet(apOS_UART_oId oId, apUART_eDTREnable *peDTRflag)

where the parameter values are:

- oId – apOS_UART_0.
- eRTSflag - apUART_DTR_ENABLED.

PL010:

```
DTR configuration tests completed successfully - informed that DTR is not supported: Pass
```

PL011:

```
DTR configuration tests completed successfully: Pass
```

5.5.5 UART_RIConfigTest

This tests the configuration of the RI output on the UART. For the PL010 variant the test checks that both functions return apERR_UNSUPPORTED, the PL011 version checks to ensure the retrieved value of the RTS configuration is the same as the set value.

The two public function calls used are:

⊕ apUART_RIConfigSet(apOS_UART_oId oId, apUART_eRIEnable eRIflag)

⊕ apUART_RIConfigGet(apOS_UART_oId oId, apUART_eRIEnable *peRIflag)

where the parameter values are:

- oId – apOS_UART_0.
- eRTSflag - apUART_RI_ENABLED.

PL010:

```
RI configuration tests completed successfully - informed that RI is not supported: Pass
```

PL011:

```
RI configuration tests completed successfully: Pass
```

5.5.6 UART_DataConfigTest

This tests the data settings and line control public functions for the UART. The test sets the UART to 8 bits, two stop bits, even parity and for PL011 full hardware control with stick parity disabled. Having done this the values are retrieved and checked against those set.

The two public function calls used are:

⊕ apUART_DataConfigSet(apOS_UART_oId oId, apUART_sConfigData *pConfig)

⊕ apUART_DataConfigGet(apOS_UART_oId oId, apUART_sConfigData *pConfig)

where the parameter values are:

- oId – apOS_UART_0.
- pConfig - apUART_EVEN_PARITY, apUART_BITS_8, apUART_TWO_STOP_BITS, apUART_HWFLOW_ALL & apUART_STICK_DISABLED.

```
Data configuration tests completed successfully: Pass
```

5.5.7 UART_ErrorIntTest

This test only runs under apUART_Version 11.

This tests the masking of the Error Interrupts. As a mask is created by ORing enumerated values of the type apUART_eErrorEnable, the test has two stages:

1. Setting and retrieving the Error Interrupt mask of BREAK errors only.
2. Setting and retrieving the Error Interrupt mask of BREAK, FRAMING, PARITY & OVERRUN by ORing the values together as indicated in the API document.

The two public function calls used are:

⊕ apUART_ErrorIntSet(apOS_UART_oId oId, UWORD32 Errormask)

⊕ apUART_ErrorIntGet(apOS_UART_oId oId, UWORD32 *pErrormask)

where the parameters are:

- oId – apOS_UART_0.
- Errormask - apUART_ERROR_INT_BREAK, apUART_ERROR_INT_OVERRUN | apUART_ERROR_INT_FRAMING | apUART_ERROR_INT_BREAK | apUART_ERROR_INT_PARITY.

```
Completed error interrupt configuration tests successfully: Pass
```

5.5.8 UART_FIFOConfigTest

This tests the masking of the FIFO interrupt tidemarks. The receive and transmit FIFO levels are set to each of the different possible values defined by the type apUART_eFIFOLevelSelect and retrieved for comparison testing. . For the PL010 variant the test checks that both functions return apERR_UNSUPPORTED, the PL011 version checks to ensure the retrieved value of the RTS configuration is the same as the set value.

The two public function calls used are:

⊕ apUART_FIFOConfigSet(apOS_UART_oId oId, apUART_sConfigFIFOs *pConfig)

⊕ apUART_FIFOConfigGet(apOS_UART_oId oId, apUART_sConfigFIFOs *pConfig)

where the parameters are:

- oId – apOS_UART_0.
- pConfig - apUART_ONEEIGHTH, apUART_SEVENEIGHTHS
  apUART_ONEQUARTER, apUART_THREEQUARTERS
  apUART_HALF, apUART_HALF
  apUART_THREEQUARTERS, apUART_ONEQUARTER
  apUART_SEVENEIGHTHS, apUART_ONEEIGHTH.

```
FIFO Configuration tests completed successfully: Pass
```

5.5.9 UART_ModemIntTest

This tests the masking of the Modem Interrupts.

For PL010 variants the test tries to set a Modem Interrupt mask for DCD interrupts only. On PL010 this level of masking is not supported so the tests expects to see apERR_UNSUPPORTED returned. The test then continues to set the Modem Interrupt mask to apUART_ModemIntEnable and retrieves the Modem Interrupt mask to check this has occurred.

For PL011 and EABBC variants the Modem Interrupt mask is created by ORing enumerated values of the type apUART_eMODEMEnable, to test this the Modem Interrupt mask of DCD and DSR interrupts is set and compared after retrieval.

The two public function calls used are:

⊕ apUART_ModemIntSet(apOS_UART_oId oId, UWORD32 Modemmask)

⊕ apUART_ModemIntGet(apOS_UART_oId oId, UWORD32 *pModemmask)

where the parameters are:

- oId – apOS_UART_0.
- Modemmask - apUART_MODEM_INT_DCD, apUART_MODEM_INT_ENABLE for PL010.
  apUART_MODEM_INT_DCD | apUART_MODEM_INT_DSR for PL011, EABBC.

```
Modem interrupt configuration test completed successfully: Pass
```

5.5.10 UART_IRConfigTest

This test only runs under apUART_Version 10 & 11.

This tests the infra-red enable and loopback settings via a set and retrieve comparison. As per the public API documented behaviour of the SIR, apUART_Disable is called before changing its value.

The public function calls used are:

⊕ apUART_Disable(apOS_UART_oId oId)

⊕ apUART_IRConfigGet(apOS_UART_oId oId, apUART_sConfigIR *pConfig)

⊕ apUART_Enable(apOS_UART_oId oId)

⊕ apUART_IRConfigGet(apOS_UART_oId oId, apUART_sConfigIR *pConfig)

where the parameters are:

- oId – apOS_UART_0
- pConfig - apUART_LOOPBACK_ENABLED, apUART_SIR_ENABLED
  apUART_LOOPBACK_DISABLED, apUART_SIR_DISABLED.

```
IR Configuration tests completed successfully: Pass
```

5.5.11 UART_PowerConfigTest

This test only runs under apUART_Version PL010 (non Integrator variant) & PL011.

This tests the setting of the power configuration options by setting the IR mode to low power and setting the low power divisor to a baud rate of 1843200. The values are then retrieved and compared.

The two public function calls used are:

⊕ apUART_PowerConfigSet(apOS_UART_oId oId, apUART_sConfigPower *pConfig)

⊕ apUART_PowerConfigGet(apOS_UART_oId oId, apUART_sConfigPower *pConfig)

where the parameters are:

- oId – apOS_UART_0.
- pConfig - apUART_MODE_LOW_POWER, 1843200.

```
Power configuration tests completed successfully: Pass
```

5.5.12 DMAConfigTest

This test only runs under apUART_Version PL011.

This tests the setting of the power configuration options by setting the IR mode to low power and setting the low power divisor to a baud rate of 1843200. The values are then retrieved and compared.

The two public function calls used are:

⊕ apUART_DMAModeSetSet(apOS_UART_oId oId, apUART_eDMAMode eDMAMode)

⊕ apUART_DMAAddressGet(apOS_UART_oId oId)

where the parameters are:

- oId – apOS_UART_0.
- eDMAMode - apUART_DMA_TX_ON

```
DMA register address 0xc0400000
DMA configuration tests completed successfully: Pass
```

5.6 UART_Test

The UART_APICallsTest suite of tests set and compare against various public API functions that set and get UART configurations. Although these go some way to checking the validity of the memory area they access, they do not test whether the UART is connected to the interrupt controller and is actually capable of sending and retrieving data. UART_Test addresses this by carrying out the transmission and receipt of text and verifying the contents on arrival. As the public test (for PL010 and PL011) cannot assume more than one UART instance, this is done under loopback.

UART_Test itself sets-up the UART for data transmission and receipt before calling a further function (UART_TestRun) with a fifteen second timeout that transmits and waits for the receipt of the data.

The public function calls used are:

⊕ apUART_Disable

⊕ apUART_TxDisable

⊕ apUART_RxDisable

⊕ apUART_FIFOConfigSet

⊕ apUART_DataConfigSet

⊕ apUART_BaudRateSet

⊕ apUART_CallbackSet

⊕ apUART_PowerConfigSet

⊕ apUART_IRConfigSet

⊕ apUART_Enable

⊕ apUART_TxEnable

⊕ apUART_RxEnable

5.6.1 UART_TestRun

This test function actually has two sections that can be run depending on the type of test set-up the user has – a terminal connection or a standalone UART (the default).

Terminal Connected

In this case the test writes to the UART (Terminal) under interrupts requesting ten characters are input. These input characters are then read from the UART (by manual polling) and re-sent to the terminal.

The public function calls used are:

⊕ apUART_Transmit(apOS_UART_oId oId, void *pBuffer, UWORD32 Size, apUART_rCallback rCallback)

⊕ apUART_Read(apOS_UART_oId oId)

with the parameters:

- oId – apOS_UART_0.
- pBuffer – UART_pMessage1, a constant array of characters requesting ten characters are entered into the terminal connected to the UART.
- Size – UART_MSGSIZE1 – size of the above array.
- rCallback – a pointer to the function UART_Written.

The function then waits for a timeout or for the user to enter ten characters and will keep polling the UART for characters. If ten characters are returned the function writes them back to the terminal under interrupts and waits for the transmit function to complete.

Single UART (default setting)

In this case the test enables loopback, sets up a receive callback and then transmits a string of size 129 bytes. This size has been chosen to not be divisible by four to check the internal word-sized buffer flushes correctly under interrupts. The public function calls used are (presented earlier, parameters not repeated):

⊕ apUART_Disable

⊕ apUART_IRConfigSet

⊕ apUART_Enable

⊕ apUART_Receive

⊕ apUART_Transmit

```
(timeout after 5 seconds)
Data transmission and reception test completed successfully: Pass
*** Pass ***
```

5.7 Results

Test run returns TRUE	YES
When **apTEST_INTERACTIVE** is FALSE, no user interaction is required	n/a (YES)
When **apTEST_VERBOSE** is FALSE, no output is produced	YES

5.7.1 Integrator variant public test run (default setting of Single UART test):

```
Memory Word Access: Pass
Memory Halfword Access: Pass
Memory Byte Access: Pass
Compiled for correct endian-ness: Pass
Memory Tests: Pass
Compiled for ARM940T - enabling caches
---------------
Testing module INT at 0x14000000
Programmed Interrupt: Pass
    *** PASS ***
Logic module 0 located
LM0 interrupt handler chained
---------------
Testing module TIMER at 0x13000000
Timer 0 test, 10 seconds: Pass
    *** PASS ***
---------------
Testing module TIMER at 0x13000100
Timer 1 test, 10 seconds: Pass
    *** PASS ***
---------------
Testing module UART at 0x16000000
Baud rate tests completed successfully: Pass
DCD configuration tests completed successfully - informed that DCD is not supported: Pass
RTS configuration tests completed successfully - informed that RTS is not supported: Pass
DTR configuration tests completed successfully - informed that DTR is not supported: Pass
RI configuration tests completed successfully - informed that RI is not supported: Pass
Data configuration tests completed successfully: Pass
FIFO Configuration tests completed successfully: Pass
Modem interrupt configuration test completed successfully: Pass
(timeout after 15 seconds)
Data transmission and reception test completed successfully: Pass
    *** PASS ***
-------------
*** Overall System:  PASS ***
```

5.7.2 Integrator variant public test run (Terminal connected setting):

```
Memory Word Access: Pass
Memory Halfword Access: Pass
Memory Byte Access: Pass
Compiled for correct endian-ness: Pass
Memory Tests: Pass
Compiled for ARM940T - enabling caches
---------------
Testing module INT at 0x14000000
Programmed Interrupt: Pass
    *** PASS ***
Logic module 0 located
LM0 interrupt handler chained
---------------
Testing module TIMER at 0x13000000
Timer 0 test, 10 seconds: Pass
    *** PASS ***
---------------
Testing module TIMER at 0x13000100
Timer 1 test, 10 seconds: Pass
    *** PASS ***
---------------
Testing module UART at 0x16000000
Baud rate tests completed successfully: Pass
DCD configuration tests completed successfully - informed that DCD is not supported: Pass
RTS configuration tests completed successfully - informed that RTS is not supported: Pass
DTR configuration tests completed successfully - informed that DTR is not supported: Pass
RI configuration tests completed successfully - informed that RI is not supported: Pass
Data configuration tests completed successfully: Pass
FIFO Configuration tests completed successfully: Pass
Modem interrupt configuration test completed successfully: Pass
(timeout after 15 seconds)
Successfully read 10 characters and transmitted them back to terminal: Pass
    *** PASS ***
-------------
*** Overall System:  PASS ***
```

5.7.3 PL011 public test run (default setting of Single UART test):

```
Memory Word Access: Pass
Memory Halfword Access: Pass
Memory Byte Access: Pass
Compiled for correct endian-ness: Pass
Memory Tests: Pass
Compiled for ARM940T - enabling caches
---------------
Testing module INT at 0x14000000
Programmed Interrupt: Pass
    *** PASS ***
Logic module 0 located
LM0 interrupt handler chained
---------------
Testing module TIMER at 0x13000000
Timer 0 test, 10 seconds: Pass
    *** PASS ***
---------------
Testing module TIMER at 0x13000100
Timer 1 test, 10 seconds: Pass
    *** PASS ***
---------------
Testing module UART at 0xc0400000
Primecell ID Registers: Pass
Baud rate tests completed successfully: Pass
DCD configuration tests completed successfully: Pass
RTS configuration tests completed successfully: Pass
DTR configuration tests completed successfully: Pass
RI configuration tests completed successfully: Pass
Data configuration tests completed successfully: Pass
Completed error interrupt configuration tests successfully: Pass
FIFO Configuration tests completed successfully: Pass
Modem interrupt configuration test completed successfully: Pass
IR Configuration tests completed successfully: Pass
Power configuration tests completed successfully: Pass
DMA register address 0xc0400000
DMA configuration test completed successfully: Pass
(timeout after 15 seconds)
Data transmission and reception test completed successfully: Pass
    *** PASS ***
-------------
*** Overall System:  PASS ***
```

6 FURTHER TESTS

6.1 Overview

This test module is named **`uarttstcalls.c`**. and is archived at **`[project]/privates/uart/`**

The project file is at the same location and is named **`testuart.mcp`**

Code coverage: Percentage of blocks tested: 59%

6.2 Outline PL011 Variant

The test module performs the following functions when the PL011 variant is selected. To perform the complete selection of tests 2 instances of the device are required linked together by null modem cable.

6.2.1 UART_APICallsTest

This function contains further calls to API functions, combined with stepping through code execution, allow the confirmation of the register set and retrieves that would be difficult to test in any other way. With various input parameters this allows for increased option coverage for the purposes of driver testing.

Unless indicated the UART affected is UART0.

UART_BaudRateTest()

This function sets and gets three different baud rates: 2400, 57600, 115200.

To confirm the baud rate is set correctly the Integer Baud Rate register (UARTIBRD) and Fractional Baud Rate register (UARTFBRD) should be checked for the value specified by the TRM formula (page 2-11).

- UARTIBRD = UART base + 0x24.
- UARTFBRD = UART base + 0x28.

```
Baud rate for UART0 (2400): 2400
Baud rate for UART0 (57600): 57600
Baud rate for UART0 (115200): 115200
```

UART_DCDTest()

This function checks that DCD configuration is supported when asserting the DCD output, and then checks the output is asserted.

To confirm the correct flag is being set the Out1 flag (bit 12) in the Control register (UARTCR) should be checked for assertion.

- UARTCR = UART base + 0x30.

```
apUART_DCDConfigSet Correctly detected that DCD is supported: Pass
apUART_DCDConfigGet Correctly detected that DCD is supported: Pass
apUART_DCDConfigGet retrieved correct value of DCD: Pass
```

UART_DataConfigTest()

This function carries out the following changes to the Line Control register (UARTLCR_H):

- 5 bits, 1 stop bit, no parity, stick parity enabled, CTS hardware flow.

- 6 bits, odd parity, RTS hardware flow control.

- 7 bits, even parity, no hardware flow control.

- 8 bits, 2 stop bits, stick parity disabled, full hardware flow control set and retrieved.

- UARTLCR_H = UART base + 0x2C (bit positions on page 3-13 of TRM should be watched for confirmation of settings).

```
apUART_DataConfigGet correctly retrieved apUART_DataConfigSet values: Pass
PL011 additional data configuration settings returned correct values: Pass
```

UART_ErrorIntTest()

This function tests the sets and gets of Error Interrupt masking. The Interrupt mask set/clear register (UARTIMSC) is altered in these tests.

Order of tests:

- Clears error interrupts (bit positions 7 – 10 are cleared).

- Framing interrupt mask set (bit position 7 set to '1').

- Parity interrupt mask set (bit position 8 set to '1').

- Break interrupt mask set and then retrieved (bit position 9 set to '1').

- Overrun interrupt mask set (bit position 10 set to '1').

- Overrun and Framing mask set, others cleared by proxy.

- Overrun, framing, break and parity interrupt mask set and retrieved.

- UARTIMSC = UART base + 0x38. Confirmation of these settings can be obtained by watching bit positions 7 – 10.

```
Testing Error interrupt setting: Pass
apUART_ErrorIntGet returned the correct value (BREAK enabled): Pass
apUART_ErrorIntGet returned the correct value: Pass
```

UART_FIFOConfigTest()

This function sets and gets the FIFO configuration settings using the Interrupt FIFO level select register (UARTIFLS).

Order of tests:

- Receive 1/8, transmit 7/8.

- Receive 1/4, transmit 3/4.

- Receive 1/2, transmit 1/2.

- Receive 3/4, transmit 1/4.

- Receive 7/8, transmit 1/8.

The settings are retrieved and compared after each set.

- UARTIFLS = UART base + 0x34. Bit positions 0 to 5 should be watched for confirmation of settings (PL011 TRM page 3-17 for details).

```
FIFOConfigGet & Set worked correctly: Pass
```

UART_ModemIntTest()

This function tests the sets and gets of the Modem Interrupt masks. The Interrupt mask set/clear register (UARTIMSC) is altered in these tests.

Order of tests:

- Set and retrieve the DCD and DSR masks (bit positions 2 and 3 set to '1').
- Disable modem interrupt detection (bits 0, 1, 2 and 3 set to '0').
- Set all the modem interrupts (bit positions 0, 1, 2 and 3 set to '1').

- UARTIMSC = UART base + 0x38. Bit positions 0 through 3 should be watched for confirmation of these settings.

```
Correctly set DCD and DSR: Pass
Read correct modem interrupt state: Pass
```

UART_IRConfigTest()

This function sets and gets the SIR and loopback setting of the UART (bit position 0 to enable/disable UART), bit position 1 (SIR enable) and bit position 7 (Loopback) in the Control register (UARTCR).

Order of tests:

- Disable UART0, enable SIR and loopback, enable UART0 (and retrieve settings).
- Disable UART0, disable SIR and loopback, enable UART0 (and retrieve settings).

- UARTCR = UART base + 0x30. Bit positions 0, 1 and 7 should be watched for confirmation of settings.

```
IRConfig tests passed: Pass
```

UART_PowerConfigTest()

This function tests the IR power saving features. The Control (UARTCR) and IrDA low-power counter (UARTILPR) registers are altered in these tests.

Low power mode (UARTCR bit position 2) and a low power baud rate of 1843200 are set and then retrieved.

To confirm the low power baud rate is set correctly the UARTILPR setting should be checked for the value specified by the PL011 TRM formula (page 3-9).

- UARTCR = UART base + 0x30. Bit position 2 should be watched for confirmation of Low Power mode (set to '1').
- UARTILPR = UART base + 0x20.

```
LPB16 value read (set as 1843200):1843200
Correctly test apUART_PowerConfigSet & apUART_PowerConfigGet: Pass
```

UART_RIConfigTest()

This function asserts the Ring Indicator (RI) output and checks the output is asserted.

- UARTCR = UART base + 0x30. To confirm the correct flag is being set the Out2 flag (bit 13) should be checked for assertion.

```
Correctly tested apUART_RIConfigSet and Get: Pass
```

UART_ReceiveTest() (**`apUART_TerminalTest`** only)

This function was created to confirm the correct behaviour of **apUART_Receive** without a word-aligned buffer and, as a side-effect, test interaction with an external serial port for a particular baud rate (57600). A timer ensures the test code does not wait forever without input. If the timer is activated before the test has completed, **UART_TerminateReceiveTimeout** is called to terminate the receive action, this sets a module-scoped flag (**UART_Timeout**) to indicate a timeout has occurred.

UART_ReceiveTest sets-up the UART with ½ FIFO interrupt levels, 8bits, one stop bit, no parity, stick and hardware flow control disabled. Although the bit positions can be checked, this functionality has been checked in earlier testing. **apUART_Receive** is called with a buffer offset by one byte.

Placing a watch on **apUART_Receive** and **apUART_IntHandler** confirms correct behaviour. One byte should be received within the word-alignment code of **apUART_Receive** and the following three bytes should be received under interrupts. The received characters' hex values are also sent to the AXD console for viewing.

Provided 'a', 'b', 'c' and 'd' are entered into the terminal:

```
Waiting for 4 chars to arrive to test buffer word alignment in UART_Receive: Pass
Read bytes numbering: 4
0x61
0x62
0x63
0x64
```

UART_WriteTest()

This function's primary purpose is to test the enabling and disabling of the FIFOs. The test initially transmits as much data as it can, (before being told the Tx FIFO is full), with FIFOs enabled (typically the amount transmitted is the transmit FIFO size in bytes plus one - 17). After a timed period to allow the data to empty, the test tries again with the FIFOs disabled (typically it now transmits one byte).

The primary bit position to check is the altering of the Line Control register's (UARTLCR_H) bit position 4 (Enable FIFOs). It should be set to '0' when disabling the FIFOs **apUART_FIFODisable(apOS_UART_0)** and set to '1' when enabling the FIFOs **apUART_FIFOEnable(apOS_UART_0)**.

- UARTLCR_H = UART base + 0x2C (bit positions 3-13 of TRM).

```
Detected that the FIFO is full: Pass
Wrote the following number of chars: 17
Write test with FIFOs disabled: Pass
Successfully detected that the FIFO is full: Pass
Wrote the following number of chars: 1
```

UART_ModemTests (**apUART_UARTTest** only)

This function tests the remaining modem signal assertions, (DCD has been tested earlier), and the detection of them under interrupts. UART1 is used to send the modem signals and UART0 detects them. Both UARTs are initialised to 57600, 8-N-1 with hardware flow control, stick parity, loopback and the SIR disabled. All the modem signals are disabled on UART1 while the modem interrupts are disabled on UART0 as part of this set-up.

The function **Listener** is registered as the **rListener** routine. Its purpose is to set a global variable "**flag**" to the modem status when a modem interrupt occurs via **apUART_ModemStatusGe**t.

Stepping through this function is not required, as each modem assertion on UART1 is checked for on UART0. The order of tests is:

- RI is asserted on UART1 and **flag** is compared against **apUART_MODEM_FLAG_RI**.

- RTS is asserted on UART1 and **flag** is compared against **apUART_MODEM_FLAG_CTS**.

- DTR is asserted on UART1, confirmed for assertion via **apUART_DTRConfigGet** and **flag** compared against **apUART_MODEM_FLAG_DSR**.

```
----Remaining Modem Tests----: Pass
RI flag detected as being set: Pass
RTS flag detected as being set (CTS flag): Pass
apUART_DTRConfigGet returned the correct value: Pass
DTR flag detected as being set (DSR flag): Pass
```

Uart_TxRxTests() (**apUART_UARTTest** only)

This function tests the **apUART_Read_N** and **apUART_Write_N** functions, the enabling and disabling of the separate receive and transmit sections and **apUART_TransmitStatusGet** and **apUART_TerminateTransmit**.

Order of events:

apUART_Read_N and **apUART_Write_N** tests

- The user listener callback is removed for UART1 by setting it to **aRNULL** – this can be confirmed by watching **UART_State[1].rNotifyListener**.

- Four bytes (“This”) are transmitted from UART0 to UART1 using **apUART_Write_N** and then received using the complementary API call **apUART_Read_N**. The received contents are compared to those transmitted.

N.B. Stepping can be undertaken during the test concentrating on **apUART_Write_N** and **apUART_Read_N**. The verification should confirm they transmit four bytes, although the transmission and reception contents are verified by the test function.

```
Successfully tested Write_N and Read_N: Pass
```

apUART_TxIntDisable and **apUART_TxIntEnable** tests

- *Confirm* **apUART_TxIntDisable** *does in fact disable the Transmit Interrupts*. To test this API call, **apUART_Transmit** is called and internally transmits one FIFO-sized worth of data and enables Transmit interrupts for further data. **apUART_TxIntDisable** is called immediately after **apUART_Transmit** – doing this should mean no further data is transmitted. The transmitted amount of data is then recorded via **apUART_TransmitStatusGet**. A wait is carried out and then **apUART_TransmitStatusGet** is called again to record the amount of data transmitted – if the two values vary, **apUART_TxIntDisable** has not worked.

```
TxIntDisable test successful: Pass
```

- *Confirm* **apUART_TxIntEnable** *does re-enable the Transmit Interrupt.* The previous test proves that disabling the Transmit Interrupt pauses the current **apUART_Transmit** action. By enabling the Transmit Interrupt briefly, by the call to **apUART_TxIntEnable** and then **apUART_TxIntDisable**, there should be a further transmission of data under interrupts. The call to **apUART_TransmitStatusGet** and comparison to the value obtained during the **apUART_TxIntDisable** test proves success if the values are different.

```
TxIntEnable test successful: Pass
```

Testing the enabling and disabling of the Receive and Transmit sections

- Receive Interrupts are disabled on UART1 (UARTIMSC register, bit positions 4 and 6).

- The transmit section of UART0 is disabled (UARTCR, bit position 9) via **apUART_TxDisable**.

- A character is written to UART0 and after a wait period UART1 attempts to read this character – a successful result is an empty receive section for UART1.

- The transmit section is then enabled on UART0 via **apUART_TxEnable**, a successful result is a non-empty receive section for UART1.

- The tests outlined above now repeat for the receive section of UART1 (**apUART_RxDisable** and **apUART_RxEnable**).

N.B. No stepping is required to confirm the above as the end result verifies the enabling and disabling of FIFOs.

```
Successfully disabled transmit FIFO: Pass
Successfully enabled the transmit FIFO: Pass
Successfully disabled receive FIFO: Pass
Successfully enabled the receive FIFO: Pass
```

apUART_TransmitStatusGet and apUART_TerminateTransmit

- Disable UART1's receive interrupts.

- Transmit some data under interrupts (greater than the FIFO size).

- Record the number of bytes written **apUART_TransmitStatusGet** and then call **apUART_TerminateTransmit**.

- Start reading some data in and counting how much has been read to free some FIFO space.

- Wait for a bit and read more data in and compare the amount read after **apUART_TerminateTransmit** to the amount transmitted before.

The test fails if the number received is greater than the recorded number sent to the FIFO before **apUART_TerminateTransmit** was called. A watch should be placed on the UART ISR (**apUART_IntHandler**) to ensure no further Transmit interrupts occur after **apUART_TerminateTransmit** is called.

```
Terminate transmit test, have already transmitted: 17
Success - apUART_TerminateTransmit stopped further data being sent: Pass
```

6.2.2 apUART_Test

This function tests the transmission and receiving of data under interrupts.

Both UARTs are reinitialised to 57600, 8 bits, no parity, 1 stop bit, SIR and loopback disabled. Additional settings (not affecting the UART operation in this case), are low power mode and a low power baud rate of 1843200. **TIMEOUT_TEST** is used to time limit this test to 20 seconds with a call to apUART_TestRun to actually perform the following tests:

apUART_TestRun

apUART_TerminalTest

This test proves out the baud rate setting to an external device. The test sends a message under interrupts to the connected terminal requesting ten characters, (the message sent is greater than the FIFO size of 16 bytes and so uses interrupts in its transmission).

Each character returned is stored in a local array using **apUART_Read** and is then re-transmitted to the terminal via **apUART_Transmit** (although interrupts are not used in this case as the message sent is less than the Tx FIFO size).

Successful execution of the above culminates with the ten characters appearing on the terminal, there is no need for any watches to demonstrate success.

```
Read 10 characters and transmitted them back to terminal: Pass
```

apUART_UARTTest

UART1 acts as the transmitter and UART0 as the receiver.

The test sends a text string of 129 bytes which is then checked to ensure it is "as sent". The size of 129 was chosen to ensure the Receive interrupt buffer flushing routines within the interrupt handler was used. Both receive and transmit action occurs under interrupts (**apUART_Receive** and **apUART_Transmit** respectively).

A successful outcome confirms the correct working of the function, however a watch can be placed on the Receive Interrupt's buffer flushing routine to confirm it executes at the end of the receive action (within **apUART_IntHandler**).

```
Successfully transmitted and received string: Pass
```

6.2.3 UART_HWFlowTest (**apUART_UARTTest** only)

This function tests the basic hardware flow capabilities of the PL011 variant through the following tests:

UART0 is used with full hardware flow control enabled and UART1 has hardware flow control disabled. UART1 is controlled within these tests to detect the settings of UART0's RTS signal (CTS on UART1), and to manipulate the behaviour of UART0 through the transmission of data and manual enabling of its own RTS signal (CTS on UART0). UART0 has its Receive Interrupt capability disabled to guarantee no receiving of any incoming data is carried out under interrupts.

Both UARTs retain their data configuration set-up in **apUART_Test** but have the additional settings of:

UART0 has full flow control enabled and a new **rListener** routine is registered for both UARTs (**UART_HWFlowTestListener**). This **rListener** routine simply assigns the status of the modem lines to a module-scoped variable when the modem status changes.

Test sections:

- *Test that a UART asserts the correct signals when empty (RTS asserted).* UART1 checks the assertion of the incoming CTS signal by checking its modem signal flag for CTS.

```
HW flow test 1 passed - UART 0 is sending RTS (CTS) signal...: Pass
```

- *Test that RTS de-asserts when a UART is sent too much data.* UART1 transmits 9 characters (more than half the FIFO tide level). This should trigger the hardware flow control to de-assert RTS on UART0 which in turn triggers a modem interrupt on UART1. A check is then carried out to see if this has in fact changed (via module-scoped variable UART1_CTSState). UART0's receive buffer is then flushed and UART1 re-checked to ensure UART0's RTS signal is re-asserted.

```
HW flow test 2 passed - UART 1 successfully detected UART 0 does not want more data: Pass
HW flow test 2 passed - flushed UART 0, it is re-sending RTS (CTS) signal...: Pass
```

- *Test that a UART only sends data from its FIFOs when receiving the correct signal (CTS asserted).* First of all RTS is de-asserted for UART1 via **apUART_RTSConfigSet** – UART0 should interpret this as UART1 being unable to accept data. A character is then written to UART0. After a delay, if **apUART_Read** on UART1 detects an empty FIFO, the test has been successful. RTS is then re-asserted,

checked for this assertion, and another **apUART_Read** carried out to confirm UART0 sends the earlier character that was waiting in UART0's Tx FIFO.

```
Successfully read RTS configuration: Pass
HW flow test 3 passed - read data once RTS was enabled: Pass
```

The final steps of the function set the **rListener** routines back to **aRNULL** for the next test, disables hardware flow control on UART0, de-assert RTS on UART1 and re-enables the Receive (and Timeout) Interrupts on UART0.

6.2.4 UART_SIRTest (**apUART_UARTTest** only)

This function aims to test the SIR and loopback capabilities of the PL011 and then the manual polling of errors. As the test board does not have an IrDA transmitter only limited testing can be carried out. The nSirOut and SirIn outputs and inputs cannot be directly connected as they expect an inverter (a role carried out by the IR device).

Test sections:

- *Test for disabled loopback works correctly*. This test disables loopback and SIR (on UART0) and **apUART_Write**s a character. An **apUART_Read** is carried out on UART0, a successful outcome being the detection of an empty FIFO to confirm loopback is not enabled.

```
Successfully detected loopback is NOT enabled (could NOT read sent char on same UART): Pass
```

- *Test loopback and SIR*. The main point of enabling both is to confirm the test register setting indicated in PL011's TRM is correct. This is the only time a test register is used under normal circumstances. The register in question is the Test Control register (UARTTCR), bit 2 (SIR Test Enable – SIRTEST0 which sets the SIR in full-duplex mode). A character is transmitted and read back on UART0 to confirm loopback is working. A watch should be set on this test register memory location to confirm correct behaviour.

```
Successfully detected loopback IS now enabled (read sent char on same Uart): Pass
```

- UARTTCR = UART base + 0x80, bit position 2 should be set to '1' after the call **apUART_IRConfigSet(apOS_UART_0, &sE)** and reset to '0' at the next call.

- *Test SIR without loopback*. Without an IR device the test sets UART0 to use the SIR while UART1 remains on Tx/Rx. Transmitting a character to UART0 and reading an empty FIFO on UART1 demonstrates success.

```
Successfully checked SIR - enabled SIR on UART 0, disabled SIR on UART 1 and could not
receive: Pass
```

- *Test the polling method of error detection*. The aim of this test is to write too many characters to UART1 and then by reading out the FIFO-size (16) number of characters from UART0 detect the overrun flag (it should be combined with the last received character). Set-up carried out is the disabling of the SIR on UART0.

```
Successfully detected overrun error via direct comparison from apUART_Read: Pass
```

- *Test correct behaviour of **apUART_TxFIFOEmpty***. The first part of this test calls this API function on UART1 expecting the return value of **TRUE** (UART1 has emptied during the previous test). The second part of the test fills up the Tx FIFO of UART1 and immediately calls **apUART_TxFIFOEmpty**, which should now return **FALSE**. Another wait occurs and a final call to **apUART_TxFIFOEmpty** is carried out to confirm it returns **TRUE** as UART1's Tx FIFO will have emptied.

```
Successful - found the TX FIFO empty: Pass
Successful - found the TX FIFO NON-empty: Pass
Successful - found the TX FIFO empty: Pass
```

6.2.5 UART_ErrorStatusTest (**apUART_UARTTest** only)

This function tests the manual polling of errors from the Error Status/Clear (UARTRSR/UARTECR) register which, although primarily included for backward compatibility, can be used with PL011. Although the register can detect Break, Parity, Framing and Overrun errors, only Overrun errors can be easily created – it is these that are used for testing.

To obtain a satisfactory basis for the tests the set-up for this function clears any existing errors (**apUART_ErrorStatusClear**), disables Receive Interrupts and removes the **rListener** (sets the **rCallback** to **aRNULL**).

Test sections:

- *Confirm* **apUART_ErrorStatusGet** *does not return an error under non-error conditions*. With the Rx FIFO empty and one character written to it and then read back, there should be no errors. To demonstrate this, a single write is made to UART1 and, after a small wait and then a read, the error status is checked on UART0. Comparing the function's return value to all the errors and not obtaining a match indicates success.

```
Success - apUART_ErrorStatusGet did NOT detect an error: Pass
```

- *Confirm the presence of an Overrun error having written (FIFOsize + 1) bytes to the Rx FIFO*. 17 characters are written to UART1 and, after a small wait, the error status of UART0 is checked. Comparing the function's return value to the Overrun flag and getting a match indicates success.

```
Success - detected the overrun immediately as per TRM: Pass
```

- *Confirm the success of clearing the error register*. After the above test **apUART_ErrorStatusClear** is called. A character is read from the FIFO and the error status of UART0 is checked. Comparing the function's return value to an Overrun error and not getting a match indicates success.

```
Success - did NOT detect the overrun error after apUART_ErrorStatusClear: Pass
```

- UARTRSR/UARTECR = UART base + 0x4, bit positions 0 to 3 (Overrun error is position 3).

A watch can be placed on the above register for UART0 during tests to confirm the behaviour of the error status register, however, care should be taken in not affecting the Data Read register and in so doing altering the number of characters within the Rx FIFO. The **List** command is a more suitable method than the memory window in allowing this.

6.2.6 UART_ContinueTest (**apUART_UARTTest** only)

This function is designed to test the API function **apUART_Continue**. This API call is designed to allow an **apUART_Receive** action to continue to wait for data after a Timeout Interrupt has occurred and informed the **rCallback** routine.

An **apUART_Receive** action is set-up with the function **ContinueCallbackTester** registered as the rCallback function. **ContinueCallbackTester** calls **apUART_Continue** having stored any received data in **UART_pReceivedOverall[]**.

The test writes one character and waits for a Timeout Interrupt to occur. The written character is then read by the **apUART_Receive** action and **ContinueCallbackTester** is called to read the number of chars transmitted (one). As the receiving array is not full, **apUART_Continue** is used to set-up the Receive & Timeout interrupts to read more data in again. Another character is transmitted to test this occurrence and then both characters transmitted are compared to the **UART_pReceivedOverall[]** characters received. Success is a match.

```
Success - apUART_Continue test worked: Pass
```

6.2.7 API Functions Called

apUART_BaudRateGet	✓	apUART_BaudRateSet	✓	apUART_CallbackSet	✓
		apUART_Continue	✓	apUART_DCDConfigGet	✓
apUART_DCDConfigSet	✓	apUART_DTRConfigGet	✓	apUART_DTRConfigSet	✓
apUART_DataConfigGet	✓	apUART_DataConfigSet	✓	apUART_Disable	✓
apUART_Enable	✓	apUART_ErrorIntGet	✓	apUART_ErrorIntSet	✓
apUART_ErrorStatusClear	✓	apUART_ErrorStatusGet	✓	apUART_FIFOConfigGet	✓
apUART_FIFOConfigSet	✓	apUART_FIFODisable	✓	apUART_FIFOEnable	✓
apUART_IRConfigGet	✓	apUART_IRConfigSet	✓	apUART_Initialize	✓
apUART_ModemIntGet	✓	apUART_ModemIntSet	✓	apUART_ModemStatusGet	✓
apUART_PowerConfigGet	✓	apUART_PowerConfigSet	✓	apUART_RIConfigGet	✓
apUART_RIConfigSet	✓	apUART_RTSConfigGet	✓	apUART_RTSConfigSet	✓
apUART_Read	✓	apUART_Read_N	✓	apUART_Receive	✓
apUART_ReceiveStatusGet	✓	apUART_RxDisable	✓	apUART_RxEnable	✓
apUART_RxIntDisable	✓	apUART_RxIntEnable	✓	apUART_TerminateReceive	✓
apUART_TerminateTransmit	✓	apUART_Transmit	✓	apUART_TransmitStatusGet	✓
apUART_TxDisable	✓	apUART_TxEnable	✓	apUART_TxFIFOEmpty	✓
apUART_TxIntDisable	✓	apUART_TxIntEnable	✓	apUART_Write	✓
apUART_Write_N	✓				

Key:

✓ - API function called within test.
× - API function not called within test.

6.2.8 Results

The following results were obtained using a single instance of the device.

```
Memory Word Access: Pass
Memory Halfword Access: Pass
Memory Byte Access: Pass
Compiled for correct endian-ness: Pass
Memory Tests: Pass

Compiled for ARM940T - enabling caches

---------------
```

```
Testing module INT at 0x14000000
Programmed Interrupt: Pass
    *** PASS ***
Logic module 0 located
LM0 interrupt handler chained
---------------
Testing module TIMER at 0x13000000
Timer 0 test, 10 seconds: Pass
    *** PASS ***
---------------
Testing module TIMER at 0x13000100
Timer 1 test, 10 seconds: Pass
    *** PASS ***
---------------
Testing module UART at 0xc0400000
Baud rate for UART0 (2400): 2400
Baud rate for UART0 (57600): 57600
Baud rate for UART0 (115200): 115200
apUART_DCDConfigSet Correctly detected that DCD is supported: Pass
apUART_DCDConfigGet Correctly detected that DCD is supported: Pass
apUART_DCDConfigGet retrieved correct value of DCD: Pass
apUART_DataConfigGet correctly retrieved apUART_DataConfigSet values: Pass
PL011 additional data configuration settings returned correct values: Pass
Testing Error interrupt setting: Pass
apUART_ErrorIntGet returned the correct value (BREAK enabled): Pass
apUART_ErrorIntGet returned the correct value: Pass
FIFOConfigGet & Set worked correctly: Pass
Correctly set DCD and DSR: Pass
Read correct modem interrupt state: Pass
IRConfig tests passed: Pass
LPB16 value read (set as 1843200):1843200
Correctly test apUART_PowerConfigSet & apUART_PowerConfigGet: Pass
Correctly tested apUART_RIConfigSet and Get: Pass
Detected that the FIFO is full: Pass
Wrote the following number of chars: 17
Write test with FIFOs disabled: Pass
Successfully detected that the FIFO is full: Pass
Wrote the following number of chars: 2
(timeout after 15 seconds)
    *** PASS ***

-------------
*** Overall System:  PASS ***
```

The following results would be obtained if two devices were available to perform a UART to UART tests

```
----Remaining Modem Tests----: Pass
RI flag detected as being set: Pass
RTS flag detected as being set (CTS flag): Pass
apUART_DTRConfigGet returned the correct value: Pass
DTR flag detected as being set (DSR flag): Pass
Successfully tested Write_N and Read_N: Pass
TxIntDisable test successful: Pass
TxIntEnable test successful: Pass
Successfully disabled transmit FIFO: Pass
Successfully enabled the transmit FIFO: Pass
Successfully disabled receive FIFO: Pass
Successfully enabled the receive FIFO: Pass
Terminate transmit test, have already transmitted: 17
Success - apUART_TerminateTransmit stopped further data being sent: Pass
(timeout after 5 seconds)
Successfully transmitted and received string: Pass
--- HW Flow control tests ---: Pass
HW flow test 1 passed - UART 0 is sending RTS (CTS) signal...: Pass
HW flow test 2 passed - UART 1 successfully detected UART 0 does not want more data: Pass
HW flow test 2 passed - flushed UART 0, it is re-sending RTS (CTS) signal...: Pass
Successfully read RTS configuration: Pass
HW flow test 3 passed - read data once RTS was enabled: Pass
Successfully detected loopback is NOT enabled (could NOT read sent char on same UART): Pass
Successfully detected loopback IS now enabled (read sent char on same Uart): Pass
Successfully checked SIR - enabled SIR on UART 0, disabled SIR on UART 1 and could not
receive: Pass
Successfully detected overrun error via direct comparison from apUART_Read: Pass
Successful - found the TX FIFO empty: Pass
Successful - found the TX FIFO NON-empty: Pass
Successful - found the TX FIFO empty: Pass
Success - apUART_ErrorStatusGet did NOT detect an error: Pass
Success - detected the overrun immediately as per TRM: Pass
Success - did NOT detect the overrun error after apUART_ErrorStatusClear: Pass
Success - apUART_Continue test worked: Pass
```

6.3 Outline PL010/Integrator AP Variant

The following set of tests indicate the reduce tests performed when PL010 or AP variant is selected:

6.3.1 UART_APICallsTest

This function contains further functions of API calls that combined with stepping through code execution allow the confirmation of the register set and retrieve functions. With various input parameters this allows for increased option coverage for the purposes of driver testing.

Unless indicated the UART affected is UART0.

UART_BaudRateTest()

Unaltered from PL011 test.

This function sets and gets three different baud rates: 2400, 57600, 115200.

To confirm the baud rate is set correctly the Integer Baud Rate registers (UARTLCR_M & UARTLCR_L) should be checked for the value specified by the TRM formula (3-8).

- UARTLCR_M = UART base + 0xC.
- UARTLCR_L = UART base + 0x10.

```
Baud rate for UART0 (2400): 2400
Baud rate for UART0 (57600): 57600
Baud rate for UART0 (115200): 115200
```

UART_DCDTest()

This function checks that DCD configuration is not supported (only supported on PL011 and EABBC) while trying to assert and check the DCD output.

```
apUART_DCDConfigSet Correctly detected that DCD is not supported: Pass
apUART_DCDConfigGet Correctly detected that DCD is not supported: Pass
```

UART_DataConfigTest()

This function carries out the following changes to the Line Control register (UARTLCR_H):

- 5 bits, 1 stop bit, no parity, stick parity enabled.
- 6 bits, odd parity.
- 7 bits, even parity.
- 8 bits, 2 stop bits set and retrieved.

- UARTLCR_H = UART base + 0x8 (bit positions on page 3-7 of the PL010 TRM should be watched for confirmation of settings).

```
apUART_DataConfigGet correctly retrieved apUART_DataConfigSet values: Pass
```

UART_ModemIntTest()

This function tests the sets and gets of the Modem Interrupt masks. The PL010 only supports the basic enabling of Modem interrupts, not the individual enabling as per the PL011 and EABBC variants. The Control register (UARTCR) bit position 3 is altered in these tests.

Order of tests:

- *Confirm a DCD mask is not supported on PL010.* **apUART_MODEM_INT_ENABLE** attempts to set a mask of DCD – success is the indication that this isn't supported. (No change to bit position 3).

- *Confirm that only the overall enabling of a modem interrupt mask is supported on PL010.* The enabling of the modem interrupts through **apUART_MODEM_INT_ENABLE** (bit 3 set to '1') is carried out and confirmed via **apUART_ModemIntGet**.

- UARTCR = UART base + 0x14. Bit position 3 should be watched for confirmation of the setting.

```
Successfully tested that DCD is not supported: Pass
Successfully tested that Modem interrupts are supported: Pass
Read correct state that modem interrupt is enabled: Pass
```

IRConfigTest()

This function ignores the IR capability, as the Integrator variant does not support it. Instead it just tests the loopback settings and the enabling and disabling of the UART. The relevant bit positions being bit position 0 (enable/disable UART) and bit position 7 (Loopback) in the Control register (UARTCR).

Order of tests:

- Disable UART0, enable loopback, enable UART0 (and retrieve settings). A character is then written to UART0 and an attempt made to read it. Success occurs by reading the character from the Rx FIFO.

- Disable UART0, disable loopback, enable UART0 (and retrieve settings). A character is then written to UART0 and an attempt made to read it. Success occurs by detecting an empty Rx FIFO.

- UARTCR = UART base + 0x14. Bit positions 0 and 7 shouild be watched for confirmation of settings.

```
Loopback test part 1: Pass
Loopback test part 2: Pass
IRConfig tests passed: Pass
```

UART_RIConfigTest()

This function attempts to assert the Ring Indicator (RI) output and confirm the message is returned that this is unsupported.

```
Correctly tested apUART_RIConfigSet - informed that RI is not supported: Pass
```

UARTReceiveTest() (apUART_TerminalTest only)

This function was created to confirm correct behaviour of **apUART_Receive** without a word-aligned buffer and, as a side effect, test interaction with an external serial port for a particular baud rate. **UARTReceiveTest** sets up the UART with 8 bits, one stop bit and no parity. Although the bit positions can be checked, this functionality has been checked in earlier testing. **apUART_Receive** is called with a buffer offset of one byte.

Placing a watch on **apUART_Receive** and **apUART_IntHandler** confirms correct behaviour. One byte should be received within the word-alignment code of **apUART_Receive** and the following three bytes should be received under interrupts. The received characters' hex values are also sent to the AXD console for viewing.

Following assumes 'a', 'b', 'c' and 'd' are entered:

```
Waiting for 4 chars to arrive to test buffer word alignment in UART_Receive: Pass
Read bytes numbering: 4
0x61
0x62
0x63
0x64
```

UART_WriteTest() (apUART_UARTTest only)

Unaltered from PL011 test.

This function's primary purpose is to test the enabling and disabling of the FIFOs. The test initially transmits as much data as it can, (before being told the Tx FIFO is full), with FIFOs enabled, (typically the amount transmitted is the transmit FIFO size in bytes plus one, 17). After a timed period to allow the data to empty, the test tries again with the FIFOs disabled (typically it now transmits one byte).

The primary bit position to check is the altering of the Line Control register's (UARTLCR_H) bit position 4 (Enable FIFOs). It should be set to '0' when disabling the FIFOs **apUART_FIFODisable(apOS_UART_0)** and set to '1' when enabling the FIFOs **apUART_FIFOEnable(apOS_UART_0)**.

- UARTLCR_H = UART base + 0x8 (bit positions 3-7 of PL010 TRM).

```
Detected that the FIFO is full: Pass
Wrote the following number of chars: 17
Write test with FIFOs disabled: Pass
Successfully detected that the FIFO is full: Pass
Wrote the following number of chars: 1
```

Uart_TxRxTests() (apUART_UARTTest only)

This function tests **apUART_Read_N** and **apUART_Write_N** and then **apUART_TxIntDisable**, **apUART_TransmitStatusGet** and **apUART_TerminateTransmit**. Both UARTs are initialised to 57600 baud, 8 bits, no parity, one stop bit and with loopback and SIR disabled.

Order of events:

apUART_Read_N and **apUART_Write_N** tests

- The user listener callback is removed for UART1 by setting it to **aRNULL** – confirm through watching **UART_State[1].rNotifyListener**.

- Four bytes ("abcd") are transmitted from UART0 to UART1 using **apUART_Write_N** and are then received using the complementary API call **apUART_Read_N**. The contents of the read are compared to the transmit.

N.B. Stepping can be useful in the test, concentrating on **apUART_Write_N** and **apUART_Read_N**. The verification should confirm the amount of data they transmit, the transmission and reception contents are verified by the test function.

```
Successfully tested Write_N and Read_N: Pass
```

apUART_TransmitStatusGet, **apUART_TerminateTransmit** & **apUART_TxIntDisable/Enable**

- *Confirm* **apUART_TxIntDisable** *does in fact disable the Transmit Interrupts*. To test this API call, **apUART_Transmit** is called which internally transmits one FIFO-sized worth of data and then enables Transmit Interrupts to transmit further data. **apUART_TxIntDisable** is called immediately after **apUART_Transmit** – doing this should mean no more data should be transmitted. The transmitted

amount of data is recorded via **apUART_TransmitStatusGet**. A wait is then initiated and **apUART_TransmitStatusGet** is used again to record the amount of data transmitted – if the two values vary, **apUART_TxIntDisable** has not worked.

```
TxIntDisable test successful: Pass
```

- *Confirm* **apUART_TxIntEnable** *does re-enable the Transmit Interrupt.* The previous test proves that disabling the Transmit Interrupt pauses the current **apUART_Transmit** action. By enabling the Transmit Interrupt briefly, by the call to **apUART_TxIntEnable** and then **apUART_TxIntDisable**, there should be a further transmission of data under interrupts. The call to **apUART_TransmitStatusGet** and comparison to the value obtained in the **apUART_TxIntDisable** test proves success if the values are different.

```
TxIntEnable test successful: Pass
```

- *Confirm* **apUART_TerminateTransmit** *does stop further transmissions*. The test in this instance is to re-enable the Transmit Interrupt and see whether further transmission still occurs. A read from UART1 after a suitable wait period should return an empty FIFO if **apUART_TerminateTransmit** is successful.

```
Success - TerminateTransmit worked - no more chars transmitted after the call: Pass
```

6.3.2 apUART_Test

(Unchanged from PL011 test except for the size of the transmitted string in the apUART_UARTTest).

This function tests the transmission and receiving of data under interrupts.

Both UARTs are reinitialised to 57600, 8 bits, no parity and 1 stop bit, SIR and loopback disabled. **TIMEOUT_TEST** is used to time limit this test to 20 seconds with a call to apUART_TestRun to actually perform the following tests:

```
UART test: requires cables to be connected (timeout after 20 seconds)
```

apUART_TestRun

apUART_TerminalTest

In this instance the test sends a message to the connected terminal requesting ten characters under interrupts, (the message sent is greater than the FIFO size of 16 bytes and so uses interrupts in its transmission).

Each character returned is stored in a local array using **apUART_Read** and then re-transmitted to the terminal via **apUART_Transmit** (although interrupts are not used in this case as the message sent is less than the Tx FIFO size).

Successful execution of the above culminating with the ten characters appearing on the terminal can be regarded as success without the need for any watches.

```
Read 10 characters and transmitted them back to terminal: Pass
```

apUART_UARTTest

UART1 acts as the transmitter and UART0 as the receiver.

The test sends a text string of 125 bytes which is then checked to ensure it is received “as sent”. The size of 125 was chosen to ensure the Receive Interrupt buffer-flushing routine within the interrupt handler was used. Both receive and transmit action occurs under interrupts (**apUART_Receive** and **apUART_Transmit** respectively).

A successful outcome confirms the correct working of the function, however a watch can be placed on the Receive interrupt’s buffer flushing routine to confirm it executes at the end of the receive action (within **apUART_IntHandler**).

```
Successfully transmitted and received string: Pass
```

6.3.3 UART_ErrorStatusTest (apUART_UARTTest only)

This function tests the manual polling of errors from the Error Status/Clear (UARTRSR/UARTECR). Although this register can detect Break, Parity, Framing and Overrun errors, only Overrun errors can be easily created – it is these that are used for testing. The second part of this function tests the **apUART_TxFIFOEmpty** API call.

To obtain a satisfactory basis for the tests the set-up for this function clears any existing errors (**apUART_ErrorStatusClear**), disables Receive Interrupts and removes the **rListener** (**aRNULL**).

Test sections:

- *Confirm* **apUART_ErrorStatusGet** *does not return an error under non-error conditions*. With the Rx FIFO empty and one character written to it and read back, there should be no errors. A single write is made to UART1 and after a small wait and a read, the error status checked. Comparing the function's return value to all the errors and not obtaining a match indicates success.

```
Success - apUART_ErrorStatusGet did NOT detect an error: Pass
```

- *Confirm the presence of an Overrun error having written (FIFOsize + 1) bytes to the Rx FIFO*. 17 characters are written to UART1 and, after a small wait, the error status of UART0 is checked. Comparing the function result to the Overrun flag and getting a match indicates success.

```
Success - detected the overrun immediately as per TRM: Pass
```

- *Confirm the success of clearing the error register*. After the above test, **apUART_ErrorStatusClear** is called. A character is read from the FIFO and the error status of UART0 is checked. Comparing the function result to an Overrun error and not getting a match indicates success.

```
Success - did NOT detect the overrun error after apUART_ErrorStatusClear: Pass
```

- UARTRSR/UARTECR = UART base + 0x4, bit positions 0 to 3 (Overrun error is position 3).

A watch can be placed on the above register for UART0 during tests to confirm the behaviour of the error status register, however, care should be taken in not affecting the Data Read register and in so doing altering the number of characters within the Rx FIFO. The **List** command is a more suitable method than the memory window.

- *Confirm* **apUART_TxFIFOEmpty** *returns* **FALSE** *when filled with some data*. UART1's Tx FIFO is filled with data and **apUART_TxFIFOEmpty** is called. A **FALSE** generates a successful result.

```
Success - TxFIFOEmpty found the FIFO non-empty after filling it: Pass
```

- *Confirm* **apUART_TxFIFOEmpty** *returns* **TRUE** *when left to empty*. UART1's Tx FIFO is given time to empty and **apUART_TxFIFOEmpty** is called. A **TRUE** generates a successful result.

```
Success - TxFIFOEmpty id'd the FIFO as empty after waiting...: Pass
```

6.3.4 UART_ContinueTest (apUART_UARTTest only)

This function is designed to test the API function **apUART_Continue**. This API call is designed to allow an **apUART_Receive** action to continue to wait for data after a Timeout Interrupt has occurred and informed the **rCallback** routine.

An **apUART_Receive** action is set-up with the function **ContinueCallbackTester** registered as the **rCallback** function. **ContinueCallbackTester** calls **apUART_Continue** having stored the received data in **UART_pReceivedOverall[]**.

The test writes one character and waits for a Timeout Interrupt to occur. The written character is then read by the **apUART_Receive** action and **ContinueCallbackTester** is called to read the number of chars transmitted (one). As the receiving array is not full, **apUART_Continue** is used to set-up the Receive & Timeout interrupts to read more data in again. Another character is transmitted to test this occurrence and then both characters transmitted are compared to the **UART_pReceivedOverall[]** characters received. Success is a match.

```
Success - apUART_Continue test worked
```

6.3.5 API Functions Called

apUART_BaudRateGet	✓	apUART_BaudRateSet	✓	apUART_CallbackSet	✓
		apUART_Continue	✓	apUART_DCDConfigGet	✓[3]
apUART_DCDConfigSet	✓[3]	apUART_DTRConfigGet	×[1]	apUART_DTRConfigSet	×[1]
apUART_DataConfigGet	✓	apUART_DataConfigSet	✓	apUART_Disable	✓
apUART_Enable	✓	apUART_ErrorIntGet	×[1]	apUART_ErrorIntSet	×[1]
apUART_ErrorStatusClear	✓	apUART_ErrorStatusGet	✓	apUART_FIFOConfigGet	×[1]
apUART_FIFOConfigSet	×[1]	apUART_FIFODisable	✓	apUART_FIFOEnable	✓
apUART_IRConfigGet	✓	apUART_IRConfigSet	✓	apUART_Initialize	✓
apUART_ModemIntGet	✓	apUART_ModemIntSet	✓	apUART_ModemStatusGet	✓
apUART_PowerConfigGet	×[2]	apUART_PowerConfigSet	×[2]	apUART_RIConfigGet	✓[3]
apUART_RIConfigSet	✓[3]	apUART_RTSConfigGet	×[1]	apUART_RTSConfigSet	×[1]
apUART_Read	✓	apUART_Read_N	✓	apUART_Receive	✓
apUART_ReceiveStatusGet	✓	apUART_RxDisable	×[1]	apUART_RxEnable	×[1]
apUART_RxIntDisable	✓	apUART_RxIntEnable	✓	apUART_TerminateReceive	✓
apUART_TerminateTransmit	✓	apUART_Transmit	✓	apUART_TransmitStatusGet	✓
apUART_TxDisable	×[1]	apUART_TxEnable	×[1]	apUART_TxFIFOEmpty	✓
apUART_TxIntDisable	✓	apUART_TxIntEnable	✓	apUART_Write	✓
apUART_Write_N	✓				

Key:

✓ - API function called within test.
× - API function not called within test.
[1]API call not directly supported on PL010.
[2]API call not directly supported on Integrator variant.
[3]API call not directly supported on PL010 – tested for apERR_UNSUPPORTED.

6.3.6 Results

UART to UART test: (apUART_UARTTest defined in uarttstcalls.c)

```
Memory Word Access: Pass
Memory Halfword Access: Pass
Memory Byte Access: Pass
Compiled for correct endian-ness: Pass
Memory Tests: Pass

Compiled for ARM940T - enabling caches

---------------
Testing module INT at 0x14000000
Programmed Interrupt: Pass
    *** PASS ***
Logic module 0 located
LM0 interrupt handler chained

---------------
Testing module TIMER at 0x13000000
Timer 0 test, 10 seconds: Pass
    *** PASS ***

---------------
Testing module TIMER at 0x13000100
Timer 1 test, 10 seconds: Pass
    *** PASS ***

---------------
Testing module UART at 0x16000000
Baud rate for UART0 (2400): 2400
Baud rate for UART0 (57600): 57600
Baud rate for UART0 (115200): 115200
apUART_DCDConfigSet Correctly detected that DCD is not supported: Pass
apUART_DCDConfigGet Correctly detected that DCD is not supported: Pass
apUART_DataConfigGet correctly retrieved apUART_DataConfigSet values: Pass
No support for error interrupts: Pass
No support for FIFO configuration test: Pass
Successfully tested that DCD is not supported: Pass
Successfully tested that Modem interrupts are supported: Pass
Read correct state that modem interrupt is enabled: Pass
Loopback test part 1 passed: Pass
Loopback test part 2 passed: Pass
IRConfig tests passed: Pass
```

```
Correctly test apUART_PowerConfigSet - informed that IrDA is not supported: Pass
Correctly test apUART_RIConfigSet - informed that RI is not supported: Pass
Detected that the FIFO is full: Pass
Wrote the following number of chars: 17
Write test with FIFOs disabled: Pass
Successfully detected that the FIFO is full: Pass
Wrote the following number of chars: 2
----Remaining Modem Tests----: Pass
Successfully tested Write_N and Read_N: Pass
TxIntDisable test successful: Pass
TxIntEnable test successful: Pass
Terminate transmit test, have already transmitted: 17
Success - apUART_TerminateTransmit stopped further data being sent: Pass
(timeout after 15 seconds)
Successfully transmitted and received string: Pass
Success - apUART_ErrorStatusGet did NOT detect an error: Pass
Success - detected the overrun immediately as per TRM: Pass
Success - did NOT detect the overrun error after apUART_ErrorStatusClear: Pass
Success - TxFIFOEmpty found the FIFO non-empty after filling it: Pass
Success - TxFIFOEmpty id'd the FIFO as empty after waiting...: Pass
Success - apUART_Continue test worked: Pass
    *** PASS ***

-------------
*** Overall System:  PASS ***
```

UART to Terminal Test : (apUART_TerminalTest defined in uarttstcalls.c)

```
Memory Word Access: Pass
Memory Halfword Access: Pass
Memory Byte Access: Pass
Compiled for correct endian-ness: Pass
Memory Tests: Pass

Compiled for ARM940T - enabling caches

---------------
Testing module INT at 0x14000000
Programmed Interrupt: Pass
    *** PASS ***
Logic module 0 located
LM0 interrupt handler chained

---------------
```

```
Testing module TIMER at 0x13000000
Timer 0 test, 10 seconds: Pass
    *** PASS ***

---------------
Testing module TIMER at 0x13000100
Timer 1 test, 10 seconds: Pass
    *** PASS ***

---------------
Testing module UART at 0x16000000
Baud rate for UART0 (2400): 2400
Baud rate for UART0 (57600): 57600
Baud rate for UART0 (115200): 115200
apUART_DCDConfigSet Correctly detected that DCD is not supported: Pass
apUART_DCDConfigGet Correctly detected that DCD is not supported: Pass
apUART_DataConfigGet correctly retrieved apUART_DataConfigSet values: Pass
No support for error interrupts: Pass
No support for FIFO configuration test: Pass
Successfully tested that DCD is not supported: Pass
Successfully tested that Modem interrupts are supported: Pass
Read correct state that modem interrupt is enabled: Pass
Loopback test part 1 passed: Pass
Loopback test part 2 passed: Pass
IRConfig tests passed: Pass
Correctly test apUART_PowerConfigSet - informed that IrDA is not supported: Pass
Correctly test apUART_RIConfigSet - informed that RI is not supported: Pass
Waiting for 4 chars to arrive to test buffer word alignment in UART_Receive: Pass
Read bytes numbering: 4
0x31
0x32
0x33
0x34
Detected that the FIFO is full: Pass
Wrote the following number of chars: 17
Write test with FIFOs disabled: Pass
Successfully detected that the FIFO is full: Pass
Wrote the following number of chars: 2
(timeout after 15 seconds)
Read 10 characters and transmitted them back to terminal: Pass
    *** PASS ***
```